Steward 2.0: Master Implementation Blueprint & Execution Spec

Architecture Specification: Deterministic Hybrid RAG, 5-Job Pipeline & Zero-Hallucination Execution

Directive: This document serves as a strict technical PRD for implementation engineers and AI coding assistants. Do not deviate into cloud-based LLM routing, generic Agentic RAG, or pure semantic search.

1. Global Architecture Constraints

- **Zero Cloud Leakage:** The entire Intent-RAG stack MUST run 100% locally on Node.js. Use in-memory tokenization, dot-product vector math, and pure TypeScript BM25.
- **No External Dependencies:** Do NOT use Pinecone, Weaviate, OpenAI embeddings, or Cohere. Use local on-device embeddings (e.g., MiniLM) and local indexing.
- **Dual-Corpus Isolation:** *Corpus A* (High-Trust: taught rules, SKU aliases, budget profiles) MUST be physically separated from *Corpus B* (Quarantined: untrusted vendor invoices, scraped deals).
- **Zero LLM Math:** The LLM is ONLY used for language understanding and record selection. It must NEVER calculate budgets, prices, or quantities. Deterministic TypeScript code handles all math and solvency logic.

2. The 5-Job Execution Pipeline

Job 1: Structured Intent Grounding (Layer 1 & 2)

Goal: Translate a vague user prompt into a strictly typed JSON payload without hallucination.

- **Alias Resolution:** Implement exact-match priority. If the user says "popular matcha", the BM25 index must resolve this to `MATCHA-CEREM-100G`.
- **Ambiguity Handling:** If a term matches two SKUs, the system MUST abstain, drop to Tier 0 (Suggest mode), and prompt the merchant.
- **Synthesis:** The LLM selects the applicable retrieved rules, but pure code populates the Zod schema. A missing field rejects the payload.
- **Budget Derivation:** Budgets are derived strictly from retrieved rules (e.g., "keep budget tight" maps to a pre-defined `DynamicAllowance` threshold), never an LLM guess.
- **Target Schema (Implement in `src/contracts/rag.ts`):**

```
export const ProposedIntentDraftSchema = z.object({
  sku: z.string(),
  targetUnits: z.number().int().positive(),
  maxBudgetUSD: z.number().positive(),
  preferredSupplierId: z.string().optional(),
  intentMode: z.enum(["URGENT_RESTOCK", "CONSERVATIVE_RESTOCK", "PRICE_DISCOVERY"
  retrievedRulesApplied: z.array(z.string())
});
```

Job 2: Deal Scout & Hybrid Retrieval (Layer 3)

Goal: Search messy supplier documents, extract data, and calculate landed costs deterministically.

- **Query Expansion:** Implement multi-query (split "best deal for 3 SKUs" into individual parallel queries) and query rewriting (e.g., "flour" → "all-purpose flour, maida").
- **Hybrid Engine:** Merge BM25 exact keyword search (with SKU-aware tokenization to keep "SKU-4421-B" intact) and dense vector search using Reciprocal Rank Fusion (RRF).
- **Reranking & MMR:** Apply a small cross-encoder reranker and Maximal Marginal Relevance (MMR) to the top-K results to ensure diversity and eliminate near-identical chunks.
- **Ingestion Pipeline:** Implement table-aware chunking (keeping pricing tier rows intact), prepend vendor/title metadata to every chunk, and enforce unit/currency normalization (e.g., mapping "12 ct" and "1 dozen" to a canonical unit).

Job 3: Pre-Flight Shield & Poison Isolation (Layer 4)

Goal: Prevent prompt injections, payee spoofing, and poisoned text from reaching the core system.

- **Taint-Aware Chunks:** Every chunk in Corpus B must carry a trust tier, provenance tag, and SHA-256 hash. Re-check hashes at generation time.
- **Quarantine:** Treat untrusted text exclusively as data literals. Run an injection-similarity detector against a known attack corpus during ingestion.
- **Payee-Swap Detection:** Compare extracted invoice bank/PayPal details against a statically verified whitelist before processing any payment holds.
- **Canary Tokens:** Embed secret canary tokens in the system prompt; if they appear in the output, trigger an immediate security halt.

Job 4: Trust Gating & Solvency (Layer 5)

Goal: Determine autonomy level based on evidence strength and check financial safety.

- **Conflict Detection:** If two retrieved sources disagree (e.g., conflicting supplier prices), the system MUST drop autonomy and require one-tap human approval.
- **Anomaly Flagging:** Flag transactions if an extracted price jumps far above the vendor's historical decision memory.
- **ZK-Vault:** Draft budgets are clamped as `min(OwnerCeiling, DynamicAllowance)`. Pass this to the on-device ZK-Vault to verify if the payroll floor is breached.

Job 5: Explainability & Execution (Layer 6)

Goal: Execute reversible holds and generate mathematically verifiable audit trails.

- **Citation Enforcement:** Every field populated in the intent payload must map directly to a chunk ID. Implement a Claim Verifier to ensure the cited chunk supports the claim.
- **Execution:** Submit the `AUTHORIZE` payload to PayPal Orders v2, enforcing a real-time 60-second Undo Window for the merchant.
- **Why-Trail:** Generate a plain-English explanation, a Dispute Dossier, and a "what changed" diff view linking directly to the quarantined source chunks.

3. Support & Implementation Matrix

Module	Required Code Implementations
Data Ingestion	PDF/HTML/Email parsers; deduplication logic; incremental indexing (adding quotes without rebuilding); freshness decay (discarding expired quotes).
Evaluation Suite	Ablation tables (BM25 vs Dense vs Hybrid); recall@k metrics; Mean Reciprocal Rank (MRR); synthetic test data generator; exact poisoning block rate testing.
Privacy & Security	Redaction layer (strips private numbers before logging); financial value-free query logging; Retrieval Inspector UI (highlights quarantined chunks in red).
Operational Extensions	Multilingual support (Hindi/Marathi parsers); Merchant feedback loop to adjust index weights; query caching for latency reduction.